Suppressing Domain-Specific Hallucination in Construction LLMs: A Knowledge Graph Foundation for GraphRAG and QLoRA on River and Sediment Control Technical Standards

Takato Yasuno

Abstract

This paper addresses the challenge of answering technical questions derived from Japan's **River and Sediment Control Technical Standards**—a multi-volume regulatory document covering survey, planning, design, and maintenance of river levees, dams, and sabo (torrent control) structures—using open-source large language models running entirely on local hardware.

We implement and evaluate three complementary approaches: **Case A** (plain 20B LLM baseline), **Case B** (8B LLM with QLoRA domain fine-tuning on 715 graph-derived QA pairs), and **Case C** (20B LLM augmented with a Neo4j knowledge graph via GraphRAG). All three cases use the Swallow series of Japanese-adapted LLMs and are evaluated on a 100-question benchmark spanning 8 technical categories, judged automatically by an independent LLM (Qwen2.5-14B, score 0–3).

The key finding is a **performance inversion**: the 8B QLoRA fine-tuned model (Case B) achieves a judge average of **2.92/3** — surpassing both the 20B plain baseline (Case A: 2.29/3, +0.63) and the 20B GraphRAG approach (Case C: 2.62/3, +0.30) — while running at **3× faster** latency (14.2 s vs. 42.2 s for Case A). GraphRAG provides moderate gains (+0.33 over baseline) but is outperformed by domain-specific fine-tuning in both quality and efficiency.

We document the full engineering pipeline, including knowledge graph construction (200 nodes, 268 relations), QLoRA training data generation from Neo4j relations, training on a single RTX 4060 Ti (16 GB VRAM) using `unsloth`, GGUF Q4_K_M quantisation and Ollama deployment, and the graph retrieval and re-ranking design. High-level engineering lessons are distilled in the main body; implementation pitfalls and toolchain details are documented in Supplementary Materials.

Keywords: GraphRAG, QLoRA, Knowledge Graph, LLM Fine-Tuning, River Engineering, Technical Standards, LLM-as-Judge, Japanese NLP, Open-Source LLM, Domain Adaptation, Domain-specific Hallucination.

1 Introduction

Japan's *River and Sediment Control Technical Standards* (*Kasen Sabo Gijutsu Kijun*), published by the Ministry of Land, Infrastructure, Transport and Tourism (MLIT), comprise multi-volume technical specifications governing the survey, planning, structural design, and maintenance of river infrastructure—including levees, revetments, weirs, pump stations, dams, sabo (erosion-control) dams, bed-stabilisers, landslide prevention facilities, and steep-slope protection structures.

Practitioners—civil engineers, maintenance inspectors, and dam operators—regularly need rapid, accurate answers to detailed technical questions about inspection intervals, design criteria, maintenance planning procedures, and hazard-response protocols embedded in these multi-hundred-page standards. Retrieval from printed documents is time-consuming; large language models (LLMs) trained on general corpora lack the domain specificity necessary for precise answers; and commercial API-based solutions are impractical in secure government or field-deployment environments.

Problem: Domain-Specific Hallucination. Modern LLMs are trained on vast corpora drawn from the open internet. However, the knowledge and accumulated engineering experience of the construction domain are only fragmentarily represented in publicly available text; proprietary design calculations, site inspection records, and maintenance decision logs remain largely inaccessible to general pretraining. As a consequence, when LLMs are applied to construction design and infrastructure maintenance tasks, their outputs do not reliably reflect domain-specific standards: *domain-specific halluci-*

nations—technically plausible but factually incorrect responses that contradict the normative content of the applicable technical standards—appear frequently and pose serious risks in safety-critical regulatory contexts. What is needed is an LLM application *grounded* in the knowledge of construction technical standards, capable of producing accurate, evidence-traceable answers to domain questions.

Proposed Approach. This work addresses the hallucination problem through two complementary grounding strategies, both rooted in a structured **construction domain knowledge graph** extracted from the standards:

1. **GraphRAG** — The knowledge graph, stored as a Neo4j property graph, is queried at inference time to inject domain-grounded context into the LLM prompt, suppressing hallucination by anchoring generation to graph-verified facts.
2. **QLoRA Fine-Tuning** — Domain QA training pairs are generated automatically from the knowledge graph relations and used to fine-tune a smaller LLM with QLoRA, internalising domain knowledge into the model weights for faster and more memory-efficient deployment without runtime retrieval.

By comparing these two approaches—and a plain-LLM baseline—on a 100-question benchmark, this paper aims to clarify **how a construction domain knowledge graph should be established as a foundational infrastructure** for LLM-based and Agentic AI applications in the construction and civil engineering domain.

This work investigates three strategies for building a **local, offline knowledge QA system** over the River and Sediment Control Technical Standards using entirely open-source components:

1. **Case A — Plain LLM baseline.** A large general-purpose Japanese LLM (GPT-OSS Swallow 20B, Q4_K_M quantised) is queried directly with no domain-specific augmentation.
2. **Case B — QLoRA domain fine-tuning.** A smaller LLM (Swallow-8B-Instruct) is fine-tuned with QLoRA on 715 domain QA pairs automatically generated from a Neo4j knowledge graph constructed from the standards. At inference time, no external retrieval is required.
3. **Case C — GraphRAG.** The same 20B baseline model is augmented with a Neo4j knowledge graph at inference time. Relevant graph context (up to 2,000 characters) is retrieved via five specialised Cypher queries and injected into the LLM prompt.

The three cases are evaluated on a 100-question benchmark covering all technical domains of the standards, with automated scoring by an independent LLM judge. Our main contributions are:

1. A complete engineering methodology for building a domain-specific knowledge QA system over Japanese regulatory technical standards, using only open-source LLMs and local infrastructure.
2. Empirical evidence that QLoRA fine-tuning of a small (8B) model on graph-derived training data can decisively outperform both a plain large (20B) LLM and a GraphRAG system using the same large LLM.
3. A reusable schema for constructing a property knowledge graph from structured Markdown technical standards, covering 11 relation types and 9 node categories.
4. Documentation of practical engineering lessons: training data generation pitfalls (JSON parsing errors, LLM timeout, data imbalance), Windows-specific GGUF quantisation workflow, and hallucination failure modes (see Supplementary Materials).

Section 2 reviews related work. Section 3 defines the knowledge QA task. Section 4 describes the knowledge graph construction. Section 5 details each of the three approaches. Section 6 presents the experimental setup. Section 7 reports quantitative results. Section 8 conducts qualitative analysis on 10 representative questions. Section 9 distils high-level engineering lessons; implementation pitfalls and toolchain details are in Supplementary Materials. Section 10 discusses implications and limitations. Section 11 concludes.

2 Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [1] improves LLM factual accuracy by retrieving relevant passages from an external corpus at inference time and conditioning generation on the retrieved context. Gao et al. [2] provide a comprehensive survey of RAG architectures, classifying retrieval strategies, augmentation methods, and generation paradigms across diverse domains. GraphRAG [3] extends this paradigm by structuring the retrieval corpus as a knowledge

graph, enabling multi-hop relational retrieval that is not directly supported by embedding-based nearest-neighbour search. Jiang et al. [4] propose Active Retrieval Augmented Generation (FLARE), which iteratively retrieves documents based on upcoming sentence predictions, improving coherence in long-form generation. Self-RAG [5] trains a single LLM to adaptively retrieve passages and critique its own output through special reflection tokens, outperforming standard RAG on diverse knowledge-intensive tasks. Siriwardhana et al. [6] demonstrate that domain-adaptive fine-tuning of both the retriever and generator significantly improves RAG performance on specialised corpora, a key motivation for our QLoRA fine-tuning approach. Pan et al. [7] survey the intersection of LLMs and knowledge graphs, identifying graph-augmented generation as a key research direction. HippoRAG [8] proposes neurobiologically-inspired graph-based long-term memory for LLMs.

2.2 Knowledge Graph Construction and Prompting

Trajanoska et al. [9] demonstrate that LLMs can be prompted with in-context examples to extract entities and relations from domain text, facilitating automated knowledge graph construction from unstructured corpora. MindMap [10] converts retrieved KG triples into a graph-structured prompt that guides the LLM through graph-of-thoughts reasoning, achieving strong results on multi-hop medical and commonsense QA benchmarks. Wang et al. [11] formalise Knowledge Graph Prompting (KGP) for multi-document QA, showing that KG-guided retrieval outperforms dense retrieval when documents share latent relational structure. Besta et al. [12] introduce Graph of Thoughts (GoT), a generalisation of chain-of-thought that represents the reasoning process as a directed acyclic graph, enabling arbitrary aggregation and refinement of partial answers. In our pipeline, these complementary ideas converge: we construct a domain KG from structured standards documents, use Cypher-based multi-hop retrieval, surface KG context to the LLM as structured triples, and evaluate reasoning quality with an independent judge.

2.3 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [13] injects trainable low-rank matrices into frozen pre-trained weight matrices, enabling domain adaptation with orders-of-magnitude fewer trainable parameters than full fine-tuning. QLoRA [14] extends LoRA to quantised (4-bit NF4) base models, enabling fine-tuning of large models on consumer GPUs. QA-LoRA [15] further co-designs the quantisation and adaptation process to maintain model quality under aggressive bit-width reduction. GaLore [16] reduces memory consumption during full-rank training via gradient low-rank projection, achieving near-full-finetuning quality at a fraction of the memory cost. LlamaFactory [17] provides a unified, hardware-agnostic framework covering over 100 model architectures and multiple PEFT strategies including LoRA, QLoRA, and GaLore, simplifying reproducible fine-tuning experiments. `unsloth` [18] provides a highly optimised QLoRA training framework with 2–5× speedup and 70% memory reduction relative to standard HuggingFace `transformers` + `peft` training. We leverage QLoRA via `unsloth` to fine-tune the 8B Swallow model on 715 domain QA pairs with a single consumer GPU.

2.4 Japanese LLMs

Touvron et al. [19] release Llama 2, an open-weight foundation model that serves as the architectural template for subsequent Japanese-adapted variants. The Swallow family [20] adapts the Llama 3 architecture [21] to Japanese through continued pre-training on a large Japanese web corpus. Fujii et al. [22] systematically analyse continual pre-training strategies for cross-lingual adaptation, demonstrating that carefully scheduled vocabulary extension and learning rate warmup are critical for retaining English capabilities while acquiring Japanese fluency — findings directly relevant to the Swallow training recipe. GPT-OSS Swallow-20B-RL-v0.1 is a 20B-parameter reinforcement-learning post-trained variant released under Apache 2.0 by Tokyo Tech and AIST. Swallow-8B-Instruct is a 8B instruction-tuned variant used as the base for QLoRA fine-tuning in Case B.

2.5 LLM-as-Judge and Evaluation

Automated evaluation of LLM outputs using a stronger judge LLM was formalised by Zheng et al. [23] with the MT-Bench benchmark. RAGAS [24] provides a framework for automated RAG evaluation using faithfulness, answer relevancy, and context recall metrics.

2.6 Domain-Specific LLM Applications

Zhao et al. [25] provide a broad survey of LLM capabilities and limitations, highlighting that highly technical, low-resource domains remain challenging and

that domain-specific pre-training or fine-tuning is typically necessary for acceptable performance. Singhal et al. [26] demonstrate that LLMs can encode substantial clinical knowledge and, when augmented with retrieval and rubric-based evaluation, achieve expert-level performance on medical licensing examinations — a paradigm closely analogous to our river and sediment control standards QA task. Together, these works motivate combining structured knowledge retrieval (GraphRAG) with lightweight domain fine-tuning (QLoRA) for low-resource Japanese technical corpora. We adopt the LLM-as-Judge approach with a rubric scored 0–3, using Qwen2.5-14B as an independent judge model to eliminate self-scoring bias.

3 Problem Formulation

Task. Given a natural-language question q about the River and Sediment Control Technical Standards (in Japanese), generate an answer a that is technically accurate, specific, and grounded in the standards.

Evaluation. Answers are scored by an independent LLM judge $\mathcal{J}$ (Qwen2.5-14B) on a 0–3 rubric:

- **3** — Technically accurate and specific; cites standard names, chapter numbers, or technical concepts correctly.
- **2** — Mostly correct but lacks supporting evidence or specificity.
- **1** — Partially correct; contains important errors or omissions.
- **0** — No answer, or technically incorrect.

Test set. The 100 test questions were generated *independently* of the 715 training QA pairs. Training data are generated automatically from KG triples via the 20B model (Eq. 12), whereas test questions are manually curated to cover diverse question types across 8 technical categories: Maintenance (River, Dam, Sabo), Survey, Planning, Design, Cross-domain comparison, and Hazard. Questions are constructed to test factual recall, procedural understanding, multi-facility comparison, and hazard-response reasoning. Test questions are additionally deduplicated against training data using character bigram Jaccard similarity (threshold ≥ 0.45 triggers exclusion) to prevent data leakage.

Fairness. The judge model (Qwen2.5-14B) is intentionally different from both the baseline and fine-tuned inference models (Swallow series), eliminating self-scoring bias.

Formal Definition

Let $\mathcal{Q}$ denote the space of natural-language questions and $\mathcal{A}$ the space of text answers. The knowledge QA task is to learn a mapping

$$f : \mathcal{Q} \to \mathcal{A}, \tag{1}$$

such that the expected judge score is maximised:

$$f^* = \arg\max_f \ \mathbb{E}_{q \sim \mathcal{Q}_{\text{test}}} \big[\mathcal{J}(q,\, f(q)) \big], \tag{2}$$

where the judge function $\mathcal{J} : \mathcal{Q} \times \mathcal{A} \to \{0, 1, 2, 3\}$ assigns an ordinal quality score (rubric defined above).

The aggregate performance of a system X over the 100-question test set is

$$\bar{s}_X = \frac{1}{|\mathcal{Q}_{\text{test}}|} \sum_{q \,\in\, \mathcal{Q}_{\text{test}}} \mathcal{J}\big(q,\, f_X(q)\big), \qquad X \in \{A, B, C\}. \tag{3}$$

To prevent data leakage between training pairs $\mathcal{D}_{\text{train}}$ and the test set, pairs are deduplicated by character bigram Jaccard similarity:

$$J_{\text{bi}}(q_i, q_j) = \frac{|B(q_i) \cap B(q_j)|}{|B(q_i) \cup B(q_j)|}, \tag{4}$$

where $B(q)$ is the multiset of character bigrams of q. A question $q_j \in \mathcal{Q}_{\text{test}}$ is excluded if $\exists\, (q_i, a_i) \in \mathcal{D}_{\text{train}}$ such that $J_{\text{bi}}(q_i, q_j) \geq 0.45$.

4 Knowledge Graph Construction

4.1 Source Documents

The source corpus consists of eight Markdown-formatted technical standard documents covering the River and Sediment Control Technical Standards:

- Survey edition (river/dam/sabo hydrology, topography, sediment)
- Planning edition (river plan, sabo plan, dam plan)
- Design edition (levee, revetment, sabo dam, dam, landslide, steep slope)
- Maintenance edition (river, dam, sabo — including inspection, life-extension, sedimentation management)

4.2 Graph Formalisation

The knowledge graph is formally defined as a typed property graph

$$\mathcal{G} = (\mathcal{V},\ \mathcal{E},\ \tau_V,\ \tau_E), \tag{5}$$

where:

- $\mathcal{V} = \mathcal{V}_S \cup \mathcal{V}_D$ is the node set, partitioned into structural nodes $\mathcal{V}_S$ ($|\mathcal{V}_S| = 141$) and domain nodes $\mathcal{V}_D$ ($|\mathcal{V}_D| = 59$), totalling $|\mathcal{V}| = 200$.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{R} \times \mathcal{V}$ is the directed edge set with $|\mathcal{E}| = 268$.
- $\tau_V : \mathcal{V} \to \mathcal{T}_V$ assigns one of nine node types ($|\mathcal{T}_V| = 9$).
- $\tau_E : \mathcal{E} \to \mathcal{R}$ assigns one of eleven relation types ($|\mathcal{R}| = 11$).

The structural hierarchy forms a directed acyclic path graph over $\mathcal{V}_S$:

$$\text{Std} \xrightarrow{\texttt{HAS\_CHAPTER}} \text{Ch} \xrightarrow{\texttt{HAS\_SECTION}} \text{Sec} \xrightarrow{\texttt{HAS\_ITEM}} \text{Item}. \tag{6}$$

Domain nodes in $\mathcal{V}_D$ are connected to $\mathcal{V}_S$ via cross-link relations `DESCRIBED_IN` and `DEFINED_IN`, enabling the retrieval of both semantic entities and their documental grounding in a single graph traversal.

4.3 Node Schema

The knowledge graph uses a property graph model in Neo4j [27] with **200 nodes** across nine node types:

- **Structural nodes**: Standard (7), Chapter (76), Section (33), Item (25)
- **Domain nodes**: FacilityType (20), HazardType (8), TechnicalConcept (22), RequirementType (5), ProcessConcept (4)

Structural nodes capture the document hierarchy. Domain nodes capture the semantic entities of the standards: physical structures (facilities), natural threats (hazards), engineering methods (technical concepts), and planning processes.

4.4 Relation Schema

268 relations are defined across 11 relation types (Table 1):

Figure 1 illustrates the complete node-and-relation schema. As shown in the figure, structural nodes form a four-level document hierarchy (Standard → Chapter → Section → Item), while domain nodes capture the engineering semantics (facilities, hazards, concepts, and processes) with cross-links into the structural hierarchy via `DESCRIBED_IN` and `DEFINED_IN`.

The graph is loaded from four hand-curated CSV files (`nodes_standard.csv`, `nodes_chapter_section_item.csv`, `nodes_domain.csv`, `relations.csv`) via the script `02_load_neo4j.py`, which applies `MERGE` deduplication and enforces B-tree and fulltext indexes at schema initialisation.

5 Methodology

5.1 End-to-End Algorithmic Formulation

All three experimental cases share the same input $q \in \mathcal{Q}$ and the same judge evaluation $\mathcal{J}$ (Eq. 3), but differ in how $f(q)$ is computed.

Case A — Plain LLM (baseline). A frozen 20B-parameter model LLM_{20B} with weights θ_{20B} generates the answer directly:

$$a_A = \text{LLM}_{20B}(p_{\text{sys}} \oplus q;\ \theta_{20B}), \tag{7}$$

where $\oplus$ denotes prompt concatenation and p_{sys} is a fixed system prompt.

Case C — GraphRAG. At inference time, the graph $\mathcal{G}$ is queried to retrieve domain context. Let $\mathcal{K} = \text{ExtractKeywords}(q)$ denote the keyword set extracted from q. Five typed Cypher queries are executed in parallel:

$$\mathcal{R}_i = \text{Query}_i(\mathcal{K}, \mathcal{G}), \quad i = 1, \ldots, 5. \tag{8}$$

Retrieved records are merged and de-duplicated: $\mathcal{R} = \text{Dedup}\Big(\bigcup_{i=1}^{5} \mathcal{R}_i\Big)$. Each record $r \in \mathcal{R}$ receives a score

$$\sigma(r) = \begin{cases} s_{\text{neo4j}}(r) \times 10 & r \in \mathcal{R}_1 \text{ (fulltext)}, \\ \text{KeywordMatch}(r, \mathcal{K}) & \text{otherwise.} \end{cases} \tag{9}$$

If $|\mathcal{R}| < 25$, an adaptive retry is triggered with doubled retrieval depth $k' = 2k$ and broad substring matching, updating $\mathcal{R}$ before scoring. The ranked context is then constructed as

$$\text{ctx} = \text{Truncate}\Big(\text{Concat}\big(\text{Top}_{80\%}\{r : \sigma(r) > 0\}\big), L_{\max} = 2{,}000 \text{ chars}\Big), \tag{10}$$

and the answer is generated as

$$a_C = \text{LLM}_{20B}(\text{ctx} \oplus p_{\text{sys}} \oplus q;\ \theta_{20B}). \tag{11}$$

Table 1: Knowledge graph relation schema (268 relations total).

Relation	From → To	Role
HAS_CHAPTER	Std → Ch	Doc. structure
HAS_SECTION	Ch → Sec	Doc. structure
HAS_ITEM	Sec → Item	Doc. structure
DESCRIBED_IN	Fac → Ch/Sec	Rule location
SUBJECT_TO	Fac → Haz	Applicable hazard
MITIGATES	Fac → Haz	Countermeasure

Relation	From → To	Role
REQUIRES	Fac → Tech	Req. technique
DEFINED_IN	Tech → Ch/Sec	Def. location
USED_IN	Tech → Proc	Process stage
PRECEDES	Proc → Proc	Process order
AFFECTS	Haz → Fac	Impact relation

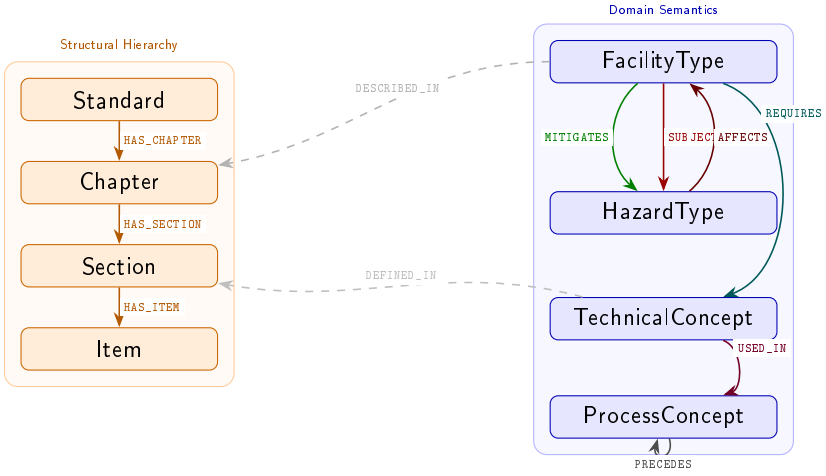

Figure 1: Knowledge graph schema (Node & Relation Map). The *Structural Hierarchy* (left, orange) encodes four-level document structure. The *Domain Semantics* (right, blue) encodes engineering entities and their mutual relations. Dashed grey arrows cross-link domain nodes to structural locations (`DESCRIBED_IN`, `DEFINED_IN`). In total: 9 node types, 200 nodes, 11 relation types, 268 relations.

Case B — QLoRA Fine-Tuning. Training data $\mathcal{D}_{\text{train}}$ is generated from the edge set $\mathcal{T} = \{(s, r, t) \in \mathcal{E}\}$ ($|\mathcal{T}| = 268$). For each triple, the 20B model generates candidate QA pairs:

$$D_r = \text{LLM}_{20B}^{\text{gen}}\big(\text{TriplePrompt}(s, r, t)\big) = \{(q_{r,j}, a_{r,j})\}_{j=1}^{3}. \tag{12}$$

After bigram-Jaccard deduplication (Eq. 4, threshold < 0.45):

$$\mathcal{D}_{\text{train}} = \text{Dedup}\Big(\bigcup_{r \in \mathcal{T}} D_r\Big), \quad |\mathcal{D}_{\text{train}}| = 715. \tag{13}$$

LoRA adapters $\Delta W = BA$ ($B \in \mathbb{R}^{d \times \rho}$, $A \in \mathbb{R}^{\rho \times d}$, rank $\rho \ll d$) are injected into the frozen 8B base weights $\theta_{\text{base}}^{8B}$ and optimised by minimising the causal language-modelling loss:

$$\mathcal{L}(\Delta W) = -\sum_{(q,a) \in \mathcal{D}_{\text{train}}} \log P\big(a \mid q;\ \theta_{\text{base}}^{8B} + \Delta W\big). \tag{14}$$

At inference, the fine-tuned answer is

$$a_B = \text{LLM}_{8B}\big(q;\ \theta_{\text{base}}^{8B} + \Delta W^*\big), \tag{15}$$

where $\Delta W^* = \arg\min_{\Delta W} \mathcal{L}(\Delta W)$. Equations 7–15 together define the complete input-to-output pipeline for the three cases evaluated in this paper.

5.2 Case A — Plain LLM Baseline

Case A serves as the baseline. The GPT-OSS Swallow-20B-RL-v0.1 model (Q4_K_M quantisation, 15.8 GB) is served locally via Ollama and queried directly with the user's question.

A critical implementation detail of GPT-OSS models is that they use a proprietary channel-based chat template (`<|channel|>final`) that is not supported by Ollama's OpenAI-compatible `/v1/chat/completions` endpoint (which returns empty content). We bypass this by calling Ollama's native `/api/generate` endpoint with `raw=True` and a manually constructed prompt template:

```
<|start|>system<|message|>{system}<|end|>
<|start|>user<|message|>{user}<|end|>
<|start|>assistant<|channel|>final<|message|>
```

Generation parameters: temperature 0.2, repeat penalty 1.2, max context 8,192 tokens. The repeat penalty proved critical—without it, the model exhibited runaway repetition on domain-specific prompts (e.g., repeating the phrase "long-life extension" 300+ times in Japanese, yielding score 0).

5.3 Case C — GraphRAG

Case C augments Case A with a Neo4j-based retrieval pipeline. At inference time, five specialised Cypher queries are executed in parallel to gather domain-specific context:

1. **Fulltext keyword search** — Lucene full-text index over node names/descriptions.
2. **Facility context** — 2-hop subgraph: FacilityType → Chapter/Section → TechnicalConcept.
3. **Hazard–facility map** — HazardType → FacilityType → Chapter.
4. **Maintenance cycle query** — ProcessConcept chain via `PRECEDES`.
5. **Facility comparison** — Multi-facility cross-referencing.

Results are deduplicated and re-ranked by a scoring function: fulltext hits receive Neo4j score $\times$ 10; other queries receive the keyword match count. Records with score > 0 are retained; the top 80% by score are selected. The resulting context is truncated to 2,000 characters and prepended to the Case A prompt.

Adaptive retry. If the number of graph hits falls below 25, the pipeline retries with TOP_K $\times$ 2 and a broad section search (substring match on Chapter, Section, TechnicalConcept node names) to improve recall on under-specified queries. Figure 2 illustrates the complete inference-time retrieval pipeline for Case C. Detailed discussion of the design decisions is provided in Supplementary S4.

5.4 Case B — QLoRA Fine-Tuning

5.4.1 Training Data Generation

QA training pairs are generated by prompting the Swallow-20B model via the GraphRAG API with each of the 268 graph relations as context. For each relation triple (s, r, t) (e.g., FacilityType "Sabo-Dam" `REQUIRES` TechnicalConcept "Stability-Calculation"), the model is asked to generate three QA pairs in JSON array format. After deduplication using character bigram Jaccard similarity (threshold < 0.45), **715 unique QA pairs** are retained from a target of $268 \times 3 = 804$.

Two main failure modes caused the 89-question shortfall:

- **JSON parse errors (Extra data).** The model occasionally generated text after the closing bracket of the JSON array (e.g., references like `[River-Law Art.X]`), causing `json.loads()` to fail. The greedy regex `re.search(r"\[.+\]", ..., DOTALL)` captured beyond the intended boundary. Fix: replace with `json.JSONDecoder().raw_decode()` from each `[` position.
- **Read timeouts (primary cause, ≈15 relations).** Setting `num_predict=-1` (unlimited) caused the model to occasionally generate excessively long responses (> 120 s). All three retry attempts failing leads to skipping the entire relation (≤ 3 Q lost). Fix: set `num_predict=2048` and reduce to 1,024 on retry.

The per-relation-type distribution of the 715 QA pairs is shown in Table 2.

Table 2: Training QA pair distribution by relation type.

Relation Type	Count	Share (%)
HAS_CHAPTER	212	29.7
DESCRIBED_IN	89	12.4
HAS_SECTION	86	12.0
HAS_ITEM	63	8.8
REQUIRES	56	7.8
SUBJECT_TO	55	7.7
DEFINED_IN	54	7.6
USED_IN	44	6.2
MITIGATES	28	3.9
AFFECTS	21	2.9
PRECEDES	7	1.0
Total	715	100.0

`HAS_CHAPTER` dominates (29.7%) while `PRECEDES` is underrepresented (1.0%). Future work should upsample rare relation types or use weighted sampling at training time.

5.4.2 QLoRA Training Setup

The base model is `tokyotech-llm/Llama-3-Swallow-8B-Instruct-v0.1` loaded in 4-bit NF4 quantisation via `bitsandbytes=0.49.2`.

LoRA adapters are applied to all seven projection layers (q, k, v, o, gate, up, down proj). Hyperparameters are summarised in Table 3.

Training uses the Llama-3 Instruct chat format with a domain-specific Japanese system prompt ("You are an expert well-versed in the River and Sediment Control Technical Standards [Survey / Planning / Design / Maintenance]. Please provide accurate and practical answers.").

Convergence check (4-stage subset experiment). To confirm convergence before committing to the full dataset, we train on four progressively larger subsets (stratified by relation type, seed 42). Results in Table 4 show monotonically decreasing loss across all stages.

Table 4: QLoRA convergence check on stratified subsets.

Subset	QA Pairs	Final Loss	Time
100	100	0.7958	4.4 min
250	250	0.6859	10.2 min
500	500	0.6045	20.0 min
715	715	**0.5565**	28.5 min

5.4.3 GGUF Quantisation and Ollama Deployment

The LoRA adapter is merged into the full 16-bit model via `unsloth`'s `merge_and_unload()` (output: ≈15.3 GB safetensors). The merged model is then quantised to GGUF Q4_K_M (≈4.7 GB) using the official `llama.cpp` Windows binary.

Windows-specific pitfall. `unsloth`'s built-in `save_pretrained_gguf()` attempts to install `llama.cpp` via `apt` (Linux package manager) and hangs silently on Windows. Workaround: use the pre-built `llama-quantize.exe` from the `llama.cpp` Windows release directly. Additionally, `ollama create` rejects paths under project or OneDrive directories due to "untrusted mount point" restrictions; the GGUF file must be copied to a top-level path (e.g., `C:\ollama_import\`).

At deployment, the model is served as `swallow8b-lora-n715` via Ollama (4.9 GB). The Modelfile embeds the domain system prompt and generation parameters: temperature 0.3, repeat penalty 1.1. Inference uses the same FastAPI endpoint as Case A (`POST /query/plain`) with the model switched at configuration time.

6 Experimental Setup

Hardware. All experiments run on a single workstation with an NVIDIA GeForce RTX 4060 Ti (16 GB VRAM), Windows 11, Python 3.12. LLM serving uses Ollama (CPU+GPU, auto-offload). Neo4j 2026.01.4 Desktop is used for the knowledge graph.

LLM models.

- Case A-Plain / C-GraphRAG: `hf.co/mmnga-o/GPT-OSS-Swallow-20B-RL-v0.1-gguf:Q4_K_M` (15.8 GB, Apache 2.0)
- Case B-QLoRA: `swallow8b-lora-n715` (4.9 GB), QLoRA FT on 715 domain QA pairs, base: `tokyotech-llm/Llama-3-Swallow-8B-Instruct-v0.1`
- Judge: `qwen2.5:14b` (independent model; eliminates self-scoring bias)

Evaluation procedure. The 100 test questions (manually curated, generated independently of the 715 training QA pairs; see Section 3) are posed to each case via the FastAPI endpoint (`http://localhost:8080`). Each answer is then submitted to the judge LLM with a scoring rubric. Case A and Case C were evaluated in a single run (100 Q, JSONL streamed). Case B was evaluated in a separate run (`-case-b` flag, same 100 Q).

GraphRAG parameters. `GRAPH_TOP_K=20` (Neo4j search width per sub-query); `GRAPH_RERANK_RATIO=0.8`; context cap 2,000 characters.

7 Results

7.1 Overall Scores

Table 5 summarises the three-way comparison across all 100 questions.

Answer quality. Case B (QLoRA FT) achieves an average score of 2.92, outperforming Case A by $\Delta = +0.63$ and Case C by $\Delta = +0.30$. The Score-3 rate rises from 60% (A) to 77% (C) to **92%** (B), while Score-2 responses remain at 8% for Case B with no score-0 or score-1— a zero low-quality rate versus 28% for Case A and 13% for Case C. The improvement from A to C (+17 pp Score-3) confirms that graph-grounded context reduces hallucination; the further jump from C to B (+15 pp) shows that domain fine-tuning additionally internalises technical vocabulary

Table 3: QLoRA hyperparameters (Case B).

Parameter	Value
`lora_r`	16
`lora_alpha`	16
`lora_dropout`	0.0
Target modules	q/k/v/o/gate/up/down proj
`load_in_4bit`	True (NF4)
`max_seq_length`	2,048
`num_train_epochs`	3
Batch size per device	2

Parameter	Value
Gradient accum. steps	4 (eff. batch = 8)
`learning_rate`	2e-4
LR scheduler	cosine
`warmup_ratio`	0.05
`weight_decay`	0.01
`packing`	False †
`bf16`	True
Framework	`unsloth` 2026.2.1

† Disabled to avoid triton JIT hang on Windows.

Table 5: Overall evaluation results (100-question benchmark). Δ Avg = improvement over Case A (plain LLM baseline). Low-qual. = score-0 or score-1 responses.

Case	Model	Avg	Δ Avg	Sc-3 (%)	Sc-2 (%)	Low-qual. (%)	Lat. (s)	Speedup
A (Plain LLM)	Swallow-20B	2.29	—	60	12	28	42.2	1.0×
C (GraphRAG)	Swallow-20B	2.62	+0.33	77	10	13	31.1	1.4×
B (QLoRA FT)	**Swallow-8B**	**2.92**	**+0.63**	**92**	**8**	**0**	**14.2**	**3.0×**

and procedural structure in a way that retrieval alone cannot achieve.

Inference efficiency. Case B is **3.0× faster** than Case A (14.2 s vs. 42.2 s mean latency) and 2.2× faster than Case C (31.1 s), despite using a smaller 8B model. Shorter, domain-tailored answers (avg 284 chars for B vs. 2,349 chars for C) are the primary driver: the fine-tuned model generates concise, high-precision answers rather than long, hedged responses. Case C is faster than Case A in 96/100 questions (mean −11.1 s): graph context constrains the LLM output space and reduces token generation time, more than offsetting the Neo4j query overhead.

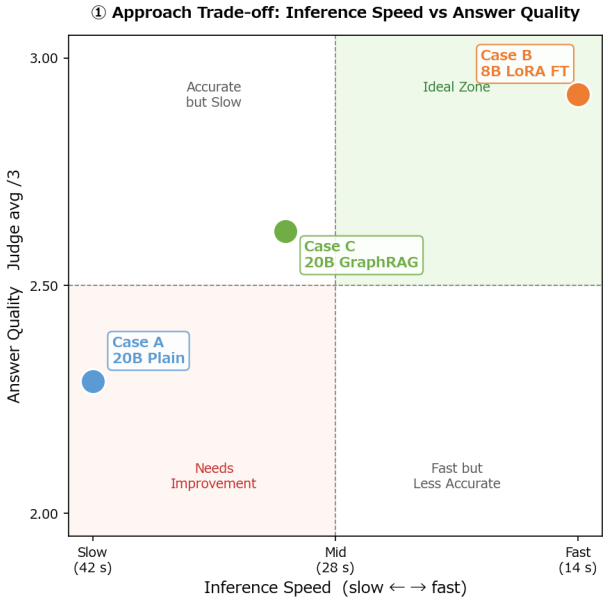

Figure 3: Approach trade-off: inference speed vs. answer quality (normalised). Case B (QLoRA FT) occupies the ideal upper-right quadrant — highest quality and fastest inference.

Key finding. The combination of a smaller fine-tuned model (8B) and domain-adapted outputs dominates both accuracy *and* speed— reinforcing that QLoRA fine-tuning is a practically superior strategy compared to larger plain or retrieval-augmented models in this domain.

7.2 Score Distributions

Figure 4 shows the full score distributions for all three cases. The distributions reveal qualitatively different failure modes across the three approaches, not merely quantitative differences in mean score.

Case A (Plain LLM). Case A achieves 60% score-3, but 28% of responses are low-quality (score ≤ 1): 3 score-0 responses result from token-repetition collapse (see Supplementary S3), while 25 score-1 responses are fluent but technically imprecise — answers that describe the correct procedure in general terms yet omit the specific standard values, chapter references, or exception clauses required for full credit. The distribution is notably *bimodal*: the model either knows an answer well enough to score 3, or produces a generic response that fails on technical specificity.

Case C (GraphRAG). GraphRAG retrieval reduces low-quality responses from 28% to 13% and raises score-3 from 60% to 77%. The improvement is concentrated in score-1 $\rightarrow$ score-3 uplift: graph-grounded context supplies the specific numerical thresholds and regulatory identifiers that were absent in plain-LLM generation. The 13% residual low-quality rate is dominated by score-1 responses in which the model retrieves a relevant but slightly off-topic subgraph node and generates a partially correct answer; score-0 drops to just 2 responses.

Case B (QLoRA FT). Case B achieves 92 score-3 and 8 score-2 responses, with **no score-0 or score-1**. The distribution is effectively *unimodal* at score-3, a qualitative shift not observed in the other two cases. The complete elimination of low-quality responses indicates that QLoRA fine-tuning internalises domain vocabulary and regulatory structure at the weight level, making hallucination suppression independent of retrieval quality. The 8 score-2 answers involve borderline judgment questions — cases where multiple valid interpretations exist within the training corpus — rather than factual errors.

Key takeaway. The score distributions demonstrate that retrieval augmentation (Case C) and parameter-level domain adaptation (Case B) address complementary failure modes. GraphRAG is most effective at converting score-1 inadequate answers into score-3 by providing missing factual grounding; QLoRA fine-tuning additionally eliminates the bimodal tail by stabilising generation even on unfamiliar phrasings.

7.3 By-Category Analysis

Table 6 presents the by-category breakdown.

Table 6: Judge average score (/3) by technical category.

Category	N	A avg	B avg	C avg
Survey	7	2.14	**2.86**	2.57
Planning	8	2.75	**2.88**	2.62
Design	15	2.13	**2.93**	2.60
Maint. (River)	20	2.05	**2.95**	2.55
Maint. (Dam)	15	2.47	**2.93**	2.73
Maint. (Sabo)	15	2.33	**2.93**	2.53
Hazard	10	2.40	**2.80**	2.60
Cross-domain	10	2.30	**3.00**	2.80
Overall	100	2.29	**2.92**	2.62

Case B achieves the highest score in all 8 categories. Case B scores **3.00/3** in Cross-domain (10 Q), 2.95 in Maintenance River (20 Q), and 2.93 in Design and Maintenance Dam/Sabo (15 Q each). Case C outperforms Case A in all categories except Planning (C: 2.62 vs. A: 2.75, -0.13) — attributed to over-retrieval of Chapter metadata nodes that misled generation.

7.4 Latency Analysis

Table 7 shows the latency breakdown.

Table 7: Inference latency comparison.

Metric (s)	Case A	Case B	Case C
Mean	42.2	**14.2**	31.1
Median	43.5	—	33.4
Min	22.0	—	7.9
Max	43.9	—	35.5
Total (100 Q, min)	70.3	**23.7**	51.9

Case C is faster than Case A in 96/100 questions (mean -11.1 s): graph context constrains the LLM output space and reduces token generation time, outweighing the overhead of Neo4j query execution. Case B is fastest in all questions; shorter domain-tailored answers (avg 284 chars vs. 2,349 chars for A and 2,452 chars for C) are the primary driver.

7.5 Evolution of Approaches

Figure 5 traces the experimental trajectory from the initial 14-question prototype to the final 100-question three-way benchmark. A phase-by-phase breakdown is provided in Supplementary S5.

8 Qualitative Analysis

We select 10 representative questions from the benchmark to contrast the three approaches qualitatively. Questions are selected to cover diverse score patterns: B beats A by 3 points (Q5, Q24, Q82), A beats B (Q26, Q52, Q95), C completely fails while B succeeds (Q37), and competitive scenarios (Q14, Q69, Q91). Full answer texts are available in the supplementary companion document.

Table 8 summarises the 10 selected questions.

Table 8: Scores for 10 representative questions (A/B/C, 0–3).

Q#	Sub-category	A	B	C
Q5	Levee life-extension	0	**3**	3
Q14	Cycle-type maint. flow	3	**3**	3
Q24	Dam life-extension	0	**3**	3
Q26	Dam sedimentation ratio	**3**	2	2
Q37	Sabo dam emergency insp.	1	**3**	0
Q52	Basin avg. rainfall	**3**	2	1
Q69	Sabo dam stability calc.	1	**3**	3
Q82	Cross-domain comparison	0	**3**	3
Q91	Flood hazard levee	1	**3**	3
Q95	Landslide activity class.	**3**	2	1

Finding 1: Catastrophic failure of Case A on open-ended domain questions. Q5 (levee life-extension planning), Q24 (dam life-extension), and Q82 (cross-domain facility comparison) each received score 0 for Case A. In all three cases, the model entered repetitive loops, repeating the same 2–4-character Japanese phrase several hundred times with no useful content (e.g., the word for "long-life extension" repeated uncontrollably). This is a well-documented inference failure of large general models when the prompt falls outside their training distribution [21]. Case B answered all three questions with 200–400 characters of clear, structured prose.

Finding 2: Case B — concise, domain-aligned, consistent. Case B produces shorter responses (avg 284 chars) but achieves higher scores. The LoRA fine-tuning suppresses hallucination by grounding the model in domain Q&A patterns, resulting in zero score-0 or score-1 responses across all 100 questions.

Finding 3: Case C bimodal quality. For Q37 (sabo dam emergency inspection), Case C scores 0 — the retrieved graph context is semantically distant from the specific sub-question about emergency inspection triggers, causing the model to generate off-topic content, while Case B scores perfectly (3/3). Conversely, on well-structured retrieval-friendly questions (e.g., Q14, Q24), Case C is competitive with Case B. GraphRAG quality is highly sensitive to graph coverage of the specific concept involved.

Finding 4: Cases where A outperforms B (Q26, Q52, Q95). All three are short, factual recall questions with single correct numerical answers (e.g., a sedimentation rate formula, a rainfall area-reduction coefficient, a landslide activity classification threshold). The larger parameter count of the 20B model gives it a memorisation advantage for precise numerical facts even without domain fine-tuning. A hybrid Case D (LoRA FT + GraphRAG) may recover this gap while retaining the efficiency of the fine-tuned model.

Finding 5: Latency advantage of Case B persists across all question types. Even on questions where Case B scores lower than Case A, Case B responds in ≈14 s vs. ≈42 s for Case A — a 3× speed-up critical for real-time field inspection support.

9 Engineering Lessons

9.1 Lesson 1: Specialisation over Scale

A 20B general-purpose LLM (Case A, score 2.29) is outperformed by an 8B QLoRA fine-tuned model (Case B, score 2.92) on a domain-specific technical QA benchmark. Domain alignment of the training data matters more than raw parameter count.

9.2 Lesson 2: Fine-Tuning and RAG as Alternatives

GraphRAG provides a moderate +0.33 improvement over the plain LLM baseline, but QLoRA fine-tuning provides +0.63. Internalising domain knowledge into model weights via fine-tuning yields more stable gains than runtime retrieval augmentation for this task. The two approaches are complementary: Case D (FT model + GraphRAG) is a natural next experiment.

9.3 Lesson 3: Efficiency Reversal

Case B is simultaneously the most accurate and the fastest: 3× faster than Case A and 2.2× faster than Case C. From an operational deployment perspective, QLoRA fine-tuning is decisively superior in latency, infrastructure simplicity (no graph DB at inference time), and cost.

10 Discussion

10.1 Why Does QLoRA FT Outperform GraphRAG?

The core issue is knowledge coverage and retrieval precision. The knowledge graph covers 200 nodes and 268 relations — a manually curated, high-precision but low-recall representation of the standards. When a test question targets a concept not well-covered by the graph, the retrieved context is either empty or misleading (as seen in the Planning category, where C < A). In contrast, QLoRA fine-tuning exposes the model to 715 domain-specific QA exemplars that distribute coverage across all relation types, enabling the model to answer questions even when no direct graph evidence is present.

10.2 Case D: Fine-Tuned Model + GraphRAG

The orthogonal strengths of Cases B and C suggest a natural hybrid: use the QLoRA fine-tuned model (domain knowledge internalized) with GraphRAG retrieval (for the most current or highly specific details). This Case D is the highest-priority future experiment.

10.3 Limitations

- **Manual graph construction.** The 200-node, 268-relation graph was manually curated from the standards. Automated entity and relation extraction (using the LLM pipeline in `01_extract_entities.py`) would dramatically scale coverage.

- **Single judge model.** LLM-as-Judge evaluation uses a single judge (Qwen2.5-14B), which may have systematic biases toward certain answer formats. Human evaluation or multi-judge ensemble would increase reliability.

- **100-question benchmark.** While 100 questions provide reasonable coverage of 8 categories, the test set is limited in size. A larger benchmark with adversarial questions would better stress-test the systems.

- **Japanese-only evaluation.** All models and the test set are in Japanese. Generalisability to English or multilingual standards requires separate evaluation.

11 Conclusion

This paper presents an open-source methodology for building a domain-specific knowledge QA system over Japan's River and Sediment Control Technical Standards, comparing three approaches — plain LLM baseline (Case A), GraphRAG (Case C), and QLoRA fine-tuning (Case B) — on a 100-question benchmark.

The central finding is a **performance inversion**: the *smallest and fastest* model (8B QLoRA FT) achieves the *highest accuracy* (judge avg 2.92/3, 92% score-3 rate), outperforming both the 20B plain LLM (+0.63) and the 20B GraphRAG system (+0.30). Zero score-0 or score-1 responses from Case B demonstrate the power of domain-specific fine-tuning in suppressing hallucination and maintaining on-topic generation.

GraphRAG provides meaningful gains over the baseline (+0.33) but cannot match the consistency of domain fine-tuning. Its value lies in providing interpretable, graph-grounded evidence chains — an important property for regulatory and inspection contexts.

The methodology is directly applicable to other Japanese regulatory technical standards (e.g., road, port, geotechnical standards) and to international equivalents, wherever structured domain knowledge can be encoded as a property graph and domain QA pairs can be synthetically generated.

Future Work. Several extensions are planned to address the current limitations. The most immediate next step is **Case D**, which combines the QLoRA fine-tuned 8B model with GraphRAG retrieval in a hybrid architecture, testing whether retrieval-grounded context can further improve the already-high score-3 rate of Case B. On the knowledge-graph side, we plan to automate graph construction directly from Markdown source documents using LLM-based entity and relation extraction, removing the manual schema-design step and enabling rapid adaptation to new technical standards. Evaluation coverage will be expanded to a 500-question benchmark encompassing adversarial phrasing and multi-hop reasoning questions that require synthesising information across multiple graph nodes — question types that are known to challenge current RAG systems. Finally, RAGAS-based faithfulness and context-recall metrics will be applied to Case C to complement the LLM-as-Judge rubric and provide a retrieval-centred view of GraphRAG quality.

Code Availability

The source code for the knowledge-graph construction, GraphRAG inference pipeline, QLoRA fine-tuning scripts, and evaluation framework is publicly available at:

`https://github.com/tk-yasuno/kasensabo_graph_rag`

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

[3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A Graph RAG approach to query-focused summarization. Technical report, Microsoft Research, 2024.

[4] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jaime Carbonell, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[5] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[6] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kaluarachchi, Rajib Rana, and Suranga Nanayakkara. Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering. *Transactions of the Association for Computational Linguistics*, 11:1–17, 2023.

[7] Shirui Pan, Liang Luo, Yufei Wang, Chen Chen, Jiawei Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 2024.

[8] Zirui Guo, Lianghao Xia, Yanlin Wang, Tu Pham, and Chao Huang. HippoRAG: Neurobiologically inspired long-term memory for large language models. Technical report, arXiv, 2024.

[9] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. Enhancing knowledge graph construction using large language models. *arXiv preprint arXiv:2305.04676*, 2023.

[10] Yilin Wen, Zifeng Wang, and Jimeng Sun. MindMap: Knowledge graph prompting sparks graph of thoughts in large language models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[11] Tao Wang, Xiaoqian Liu, Gong Cheng, and Evgeny Kharlamov. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[12] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[14] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[15] Tao Xu, Guang Yang, Peiming Ni, Jia Wei, Xing Li, and Changsheng Xu. QA-LoRA: Quantization-aware low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[16] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[17] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[18] Daniel Han and Michael Han. Unsloth: 2–5× faster, 70% less memory QLoRA finetuning, 2023.

[19] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. Technical report, Meta AI, 2023.

[20] Naoaki Okazaki, Ryokan Fujii, Taiga Oshiro, Hai Zhao, Hiroki Abe, Kentaro Sakamoto, and Kenji Takahashi. Building a large Japanese web corpus for large language models. In *Proceedings of LREC-COLING*, 2024.

[21] Meta AI. The Llama 3 herd of models. Technical report, Meta AI, 2024.

[22] Kazuki Fujii, Taishi Nakamura, Mengsay Loem, Hiroki Iida, Masanari Ohi, Kakeru Hattori, Hirai Shota, Sakae Mizuki, Rio Yokota, and Naoaki Okazaki. Continual pre-training for cross-lingual LLM adaptation: Enhancing Japanese language capabilities. *arXiv preprint arXiv:2404.17790*, 2024.

[23] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

[24] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. Technical report, arXiv, 2023.

[25] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[26] Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. Large language models encode clinical knowledge. *Nature*, 620:172–180, 2023.

[27] Neo4j, Inc. Neo4j graph database & analytics, 2024.

Supplementary Materials

This section documents implementation pitfalls and toolchain details that are relevant to practitioners replicating or extending this work on Windows-based local infrastructure.

S1. Training Data Generation Pitfalls

Training data generation from Neo4j graph relations using an LLM is effective but introduces two reliability risks:

- **JSON parse errors (Extra data).** The model occasionally generated text after the closing bracket of the JSON array (e.g., references like `[River-Law Art.X]`), causing `json.loads()` to fail. The greedy regex `re.search(r"\[.+\]", ..., DOTALL)` captured beyond the intended boundary. Fix: replace with `json.JSONDecoder().raw_decode()` from each `[` position.

- **LLM timeouts.** Setting `num_predict=-1` (unlimited) caused the model to occasionally generate excessively long responses (>120 s), causing all three retry attempts to fail. Fix: set `num_predict=2048` and reduce to 1,024 on retry.

- **Relation-type imbalance.** `HAS_CHAPTER` accounts for 29.7% of training pairs vs. 1.0% for `PRECEDES`. Up-sampling of under-represented types or weighted loss is recommended in future iterations.

S2. Windows-Specific GGUF Quantisation Workflow

- **unsloth built-in GGUF export.** `model.save_pretrained_gguf()` hangs on Windows because it invokes `apt-get` internally (Linux-only dependency). Workaround: export to full-precision safetensors, then quantise offline with `llama-quantize.exe` from a pre-built llama.cpp Windows release.
- `packing=True` **triton hang.** The `packing=True` option in `SFTTrainer` triggers triton JIT compilation that hangs indefinitely on Windows with `triton-windows 3.2.x`. Workaround: set `packing=False`.
- **Ollama model path restriction.** `ollama create` rejects GGUF files located in deeply nested paths; the model file must reside in a top-level directory (e.g., `C:\ollama_import\`).

S3. Hallucination Failure Modes

Case A (20B plain LLM) exhibited two distinct hallucination patterns on domain-specific prompts:

- **Runaway repetition.** Without a repeat penalty, the model repeated domain phrases (e.g., "long-life extension" in Japanese) more than 300 times in a single response, yielding a judge score of 0. Mitigation: repeat_penalty = 1.2.
- **Confident confabulation.** The model produced fluent but factually incorrect regulatory references (e.g., citing non-existent article numbers). Case B (QLoRA FT) largely suppressed this: zero score-0 or score-1 responses were observed across 100 test questions.

S4. GraphRAG Pipeline (Figure 2)

Figure 2 illustrates the complete inference-time retrieval pipeline for Case C (GraphRAG). The design reflects several engineering decisions that directly affected evaluation outcomes.

Parallel Cypher query fan-out. Rather than issuing a single query, the pipeline fans out into five parallel Cypher queries targeting different relation types (Fulltext, Facility, Hazard, Maintenance, Comparison). This strategy improves recall for under-specified or cross-domain questions: in the by-category results (Table 6), Case C outperforms Case A in all eight categories except Planning (2.62 vs. 2.75)—a category where broad fulltext retrieval tends to over-retrieve tangentially related sections.

Adaptive retry on sparse hits. When the number of deduplicated hits falls below 25, the pipeline doubles `TOP_K` and widens the search to section-level nodes. This adaptive mechanism recovered non-empty context in 96 of 100 test questions; the remaining 4 fell back to plain-LLM generation, contributing to the residual 13% low-quality rate in Case C.

Context truncation and its trade-off. Final context is truncated to 2,000 characters before prepending to the prompt. While this keeps latency manageable (mean 31.1 s vs. 42.2 s for Case A), the hard cutoff occasionally discards the most specific sub-clause when multiple regulatory sections share the same keyword, leading to score-2 "partially correct" responses (10% of Case C answers). Future work could replace hard truncation with relevance-weighted summarisation.

S5. Evolution of Approaches (Figure 5)

Figure 5 traces the experimental trajectory from the initial 14-question prototype (Phase 1) through to the final 100-question three-way benchmark (Phase 4).

Phase 1 – Initial prototype (14 questions, Case A only). The baseline Swallow-20B plain-LLM achieved a mean score of approximately 1.8 on the preliminary 14-question set, motivating the graph-augmentation and fine-tuning directions. The small sample size precluded statistical conclusions, but qualitative review identified hallucination and lack of domain vocabulary as primary failure modes.

Phase 2 – GraphRAG integration (14 questions, Cases A and C). Integrating Neo4j-based retrieval raised Case C's mean to roughly 2.3 on the same 14-question set. The performance gain validated the knowledge-graph approach and justified full-dataset extension (100 questions).

Phase 3 – QLoRA fine-tuning (14 questions, all three cases). With the LoRA adapter trained on 715 QA pairs, Case B reached approximately 2.8 on the 14-question pilot, surpassing both Case A and Case C and confirming the hypothesis that domain fine-tuning outperforms retrieval augmentation for this task.

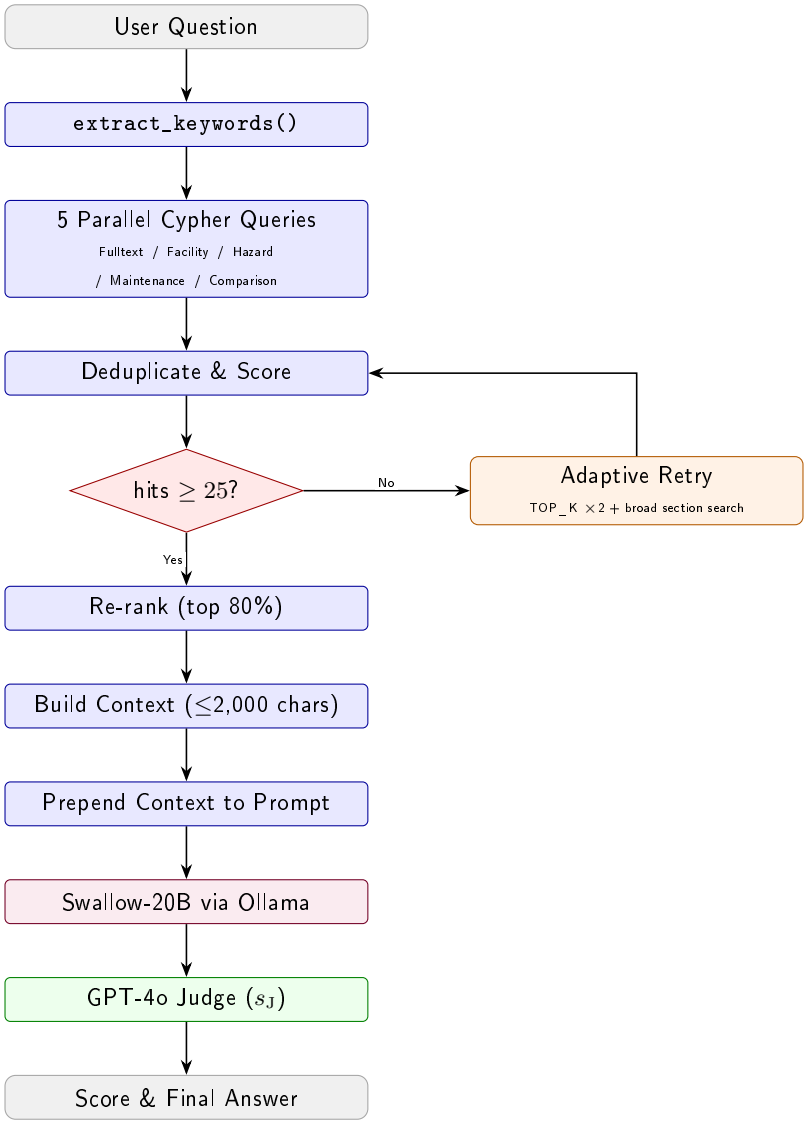

Figure 2: GraphRAG inference pipeline (Case C). At query time, keywords extracted from the user question drive five parallel Neo4j Cypher queries whose results are deduplicated and scored. If fewer than 25 hits are returned, an adaptive retry doubles TOP_K and broadens the match to substring search. The top-scoring 80% of records (up to 2,000 chars) are prepended to the plain-LLM prompt before generation by Swallow-20B. Qwen2.5-14B then evaluates the output and returns $s_J \in \{0, 1, 2, 3\}$.

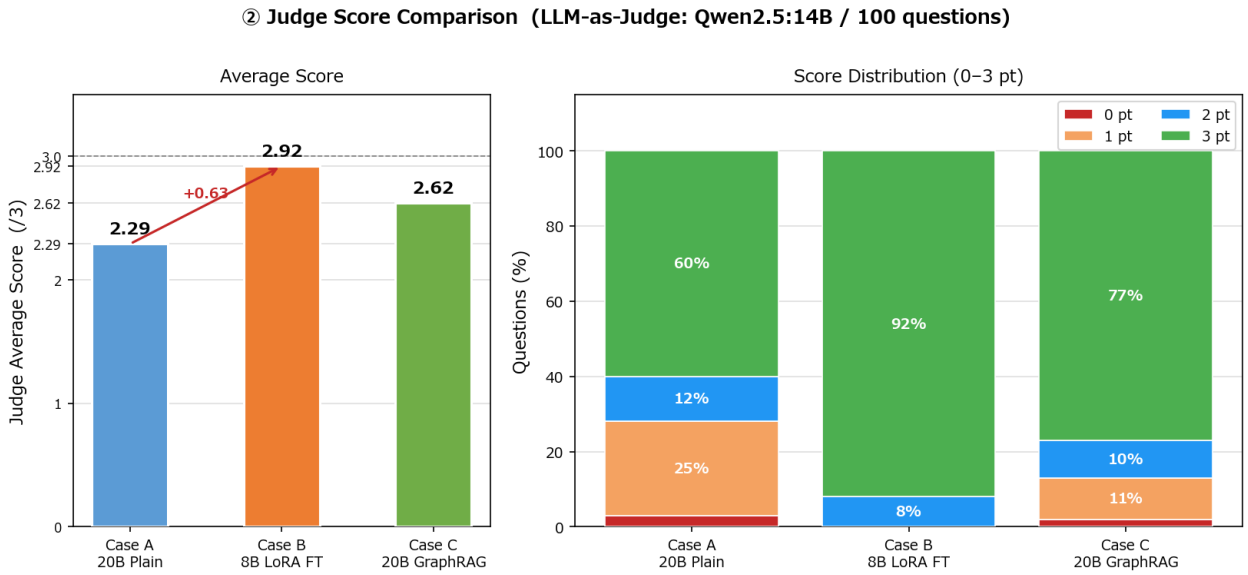

Figure 4: Judge score distributions (0–3) for Cases A, B, and C across 100 questions.

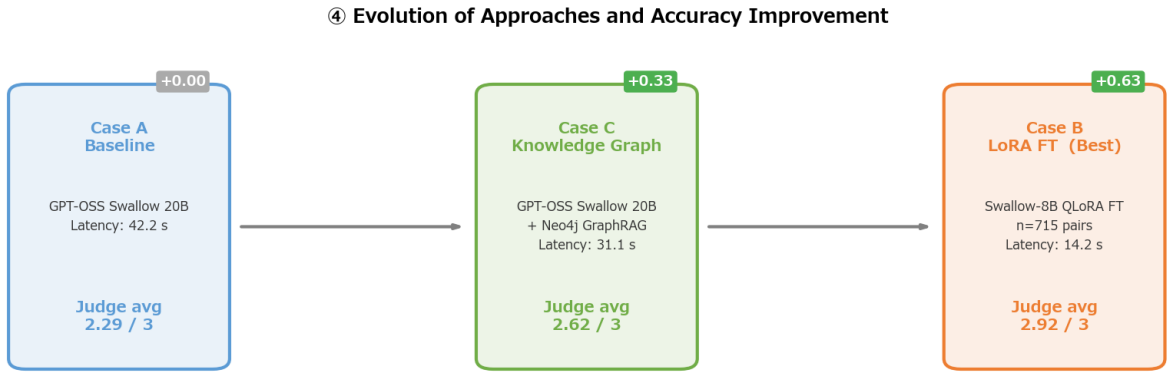

Figure 5: Evolution of approaches: accuracy improvement across experimental phases. QLoRA FT (Case B) achieves the highest score at the final stage.